

UNIT – II NOTES

Policy evaluation, policy improvement, policy iteration, and value iteration algorithms.

1. Introduction

Reinforcement Learning (RL) is a machine learning paradigm where an **agent** learns to make decisions by interacting with an **environment**. The objective is to learn an **optimal policy** that maximizes the cumulative future reward.

An RL problem is usually represented using a **Markov Decision Process (MDP)**.

An MDP consists of:

- **S** – Set of states
- **A** – Set of actions
- **P(s'|s,a)** – State transition probability
- **R(s,a,s')** – Reward function
- **γ (Gamma)** – Discount factor ($0 \leq \gamma \leq 1$)

Important Definitions

Policy (π)

A policy defines the behavior of an agent.

It specifies which action should be taken in each state.

Deterministic Policy

$$\pi(s) = a$$

One action is selected.

Stochastic Policy

$$\pi(a | s) = P(A = a | S = s)$$

Action is selected according to probabilities.

State Value Function

The expected return when starting from state **s** and following policy **π**.

$$V^{\pi}(s) = E_{\pi}[G_t \mid S_t = s]$$

where

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Action Value Function

Expected return when taking action **a** in state **s**.

$$Q^{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a]$$

Bellman Expectation Equation

The value of a state equals the immediate reward plus the discounted value of future states.

$$V^{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s', r} P(s', r \mid sa)[r + \gamma V^{\pi}(s')]$$

This equation forms the basis of Policy Evaluation.

2. Policy Evaluation

Definition

Policy Evaluation computes the value function of a **given policy**.

In simple words,

"How good is this policy?"

The policy remains fixed.

Only the values of states are updated.

Objective

Given

- Policy π

Find

$$V^\pi(s)$$

for every state.

Bellman Expectation Update

$$V_{k+1}(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | sa) [R(s, a, s') + \gamma V_k(s')]$$

Repeated until convergence.

Policy Evaluation Algorithm

Step 1

Initialize

$$V(s) = 0$$

for all states.

Step 2

For each state

Calculate

Expected reward

-

Discounted value of next state

Step 3

Update every state value.

Step 4

Repeat until

$$|V_{new}(s) - V_{old}(s)| < \theta$$

where θ is a very small threshold.

Pseudocode

Initialize $V(s)=0$

Repeat

$$\Delta = 0$$

For each state s

$$v = V(s)$$

$$V(s) = \sum \pi(a|s)$$

$$\sum P(s'|s,a)$$

$$[R + \gamma V(s')]$$

$$\Delta = \max(\Delta, |v - V(s)|)$$

Until $\Delta < \theta$

Return V

Example

Suppose

State A

Action Right

Reward = 5

Next State Value = 10

Discount

$\gamma = 0.9$

Then

$$\begin{aligned} V(A) &= 5 + 0.9(10) \\ &= 14 \end{aligned}$$

Advantages

- Gives exact value of a policy
 - Stable
 - Foundation for Policy Iteration
-

Disadvantages

- Requires complete model

- Slow for large state spaces
-

3. Policy Improvement

Definition

Once state values are known,

Improve the policy by choosing the best action.

Question answered:

"Can we choose a better action?"

Greedy Policy

Choose action with maximum action value.

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

Bellman Optimality Equation

$$Q(s, a) = \sum_{s'} P(s' | sa) [R + \gamma V(s')]$$

Choose

$$\arg \max_a Q(s, a)$$

Policy Improvement Algorithm

1. Evaluate current policy.
2. Compute Q for every action.
3. Select action with maximum Q.
4. Replace old policy.

5. Repeat.

Example

Current State

Possible actions

Action Value

Up 15

Down 18

Left 10

Right 22

Best action

Right

Policy becomes

$$\pi(s) = \textit{Right}$$

Policy Improvement Theorem

If

$$Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$$

for all states,

then

$$V^{\pi'}(s) \geq V^{\pi}(s)$$

Meaning

New policy is always equal or better.

4. Policy Iteration

Definition

Policy Iteration alternates between

- Policy Evaluation
- Policy Improvement

until policy becomes optimal.

Workflow

Initial Policy

↓

Policy Evaluation

↓

Policy Improvement

↓

Policy Stable?

No → Repeat

Yes

Optimal Policy

Algorithm

Step 1

Initialize random policy.

Step 2

Evaluate policy.

Step 3

Improve policy.

Step 4

If policy changes

Go back to evaluation.

Otherwise stop.

Pseudocode

Initialize policy π

Repeat

 Evaluate π

 policyStable=True

For every state

$\text{oldAction} = \pi(s)$

$\pi(s) = \operatorname{argmax}_a Q(s, a)$

if $\text{oldAction} \neq \pi(s)$

$\text{policyStable} = \text{False}$

Until policyStable

Return π

Example

Iteration 1

Random policy

↓

Evaluate values

↓

Improve actions

↓

New policy

↓

Evaluate again

↓

Improve again

↓

Optimal policy

Advantages

- Converges quickly
 - Produces exact optimal policy
 - Fewer iterations than Value Iteration
-

Disadvantages

- Policy evaluation may be expensive
 - Large memory usage
-

5. Value Iteration

Definition

Value Iteration combines Policy Evaluation and Policy Improvement into one update.

Instead of fully evaluating a policy, it directly updates values using the Bellman Optimality Equation.

Bellman Optimality Equation

$$V(s) = \max_a \sum_{s'} P(s' | sa) [R + \gamma V(s')]$$

Unlike Policy Evaluation, the **max** operator is used.

Value Iteration Algorithm

Step 1

Initialize

$$V(s) = 0$$

Step 2

For every state

Calculate

Maximum expected return.

Step 3

Update values.

Step 4

Repeat until convergence.

Step 5

Extract policy

$$\pi(s) = \arg \max_a \sum_{s'} P(s' | sa) [R + \gamma V(s')]$$

Pseudocode

Initialize $V(s)=0$

Repeat

$$\Delta=0$$

For each state

$$v=V(s)$$

$$V(s)=\max_a \sum P(s'|s,a)$$

$$[R+\gamma V(s')]$$

$$\Delta=\max(\Delta, |v-V(s)|)$$

Until $\Delta < \theta$

Return V

Extract Policy

Example

State A

Action Return

Up 15

Down 12

Left 18

Right 25

Maximum

25

Hence

$$V(A) = 25$$

Advantages

- Simpler than Policy Iteration
- Requires less memory
- Faster when approximate solutions are acceptable

Disadvantages

- More iterations may be required
- Values converge slowly when γ is close to 1

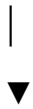
Policy Iteration vs Value Iteration

Feature	Policy Iteration	Value Iteration
Starts with	Random policy	Random values
Policy Evaluation	Full evaluation	Partial (one-step update)
Policy Improvement	Separate step	Embedded in value update
Convergence	Usually fewer iterations	Usually more iterations
Computation per Iteration	Higher	Lower
Memory Requirement	Higher	Lower
Output	Optimal policy	Optimal value function, then policy

Feature	Policy Iteration	Value Iteration
Best for	Small and medium MDPs	Large MDPs and approximate planning

Relationship Among the Algorithms

Start



Random Policy π



Policy Evaluation

Compute $V_{\pi}(s)$ using
Bellman Expectation Equation

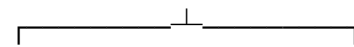


Policy Improvement

$\pi'(s) = \arg\max_a Q_{\pi}(s, a)$



Policy Changed?



Yes



No



Repeat Evaluation and Improvement
(Policy Iteration)

Value Iteration

Initialize $V(s)=0$



Bellman Optimality Update

$V(s)=\max_a[R+\gamma V(s')]$



Repeat Until Convergence



Extract Policy

$$\pi(s) = \operatorname{argmax}_a [R + \gamma V(s')]$$

Key Differences

Aspect	Policy Evaluation	Policy Improvement	Policy Iteration	Value Iteration
Purpose	Evaluate a fixed policy	Improve a policy	Find optimal policy	Find optimal value function
Uses Policy	Fixed	Updated	Yes	No (until final extraction)
Bellman Equation	Expectation	Greedy improvement using Q-values	Expectation + Improvement	Optimality
Stops When	Values converge	Policy improves	Policy becomes stable	Values converge
Output	State values	Better policy	Optimal policy	Optimal values and derived policy

Applications

- **Robot Navigation:** Determining the shortest or safest path in a grid world.
- **Autonomous Vehicles:** Choosing optimal driving actions in changing traffic conditions.

- **Game Playing:** Finding winning strategies in games such as chess or Go.
- **Inventory Management:** Optimizing stock replenishment decisions.
- **Healthcare:** Recommending treatment policies over time.
- **Finance:** Portfolio optimization and trading strategies.
- **Industrial Automation:** Controlling manufacturing processes efficiently.